SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I Shaik Mohammed Waseem Rayan (192373004) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Shaik Mohammed Waseem Rayan

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- Explain how Hill Climbing would work in this scenario and identify its limitations
- Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- Explain why this problem requires an online search agent instead of offline search

- Analyze the challenges of partial observability and dynamic environments
- Describe how the rover builds and updates its internal model
- Compare online search with uninformed and informed search strategies
- Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- Clearly define variables, domains, and constraints with explanation
- Explain how backtracking search works in this scenario
- Discuss how constraint propagation (e.g., forward checking) improves efficiency
- Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- Explain how the Minimax algorithm models this problem
- Analyze the computational challenges of large game trees
- Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- Design an evaluation function and justify the factors considered
- Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

SCENARIO 1: AI-Powered Emergency Medicine Delivery Drone

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.

Greedy Best-First Search (GBFS) is an informed search algorithm that selects the next node based only on the heuristic value **h(n)**, which estimates the remaining distance to the goal. In this scenario, the heuristic can be the straight-line distance from the drone's current position to the destination. The algorithm always chooses the path that appears closest to the destination without considering the distance already travelled or the total travel cost.

Initially, the drone calculates the estimated distance to the destination from all neighboring locations. It immediately moves toward the node with the smallest heuristic value. If the drone later discovers that a road is flooded or blocked, it backtracks and searches for another nearby path with the next lowest heuristic value.

Strengths

- Makes decisions very quickly, which is important during medical emergencies.
- Requires less computation than many other informed search algorithms.
- Suitable when a fast response is more important than finding the perfect path.
- Performs well when the heuristic accurately reflects the actual distance.

Weaknesses

- Ignores the distance already travelled, often resulting in longer overall routes.
- Can become trapped in dead ends or blocked regions.
- May repeatedly change direction when new obstacles are detected.
- Does not guarantee the shortest or safest path.
- Performs poorly in highly dynamic environments with frequent changes.

Real-world Interpretation

Suppose a drone is delivering insulin during severe flooding. Greedy Best-First Search may choose the road that appears geographically closest to the hospital. However, after flying halfway, it discovers that flooding has destroyed a bridge. The drone must return and search for another path, wasting valuable time.

Therefore, although GBFS provides rapid decisions, it is unreliable when the environment changes continuously.

*ii. Explain how A Search would handle the same situation, including how it balances cost and heuristic.**

A* Search is an informed search algorithm that combines the actual travel cost with the estimated remaining cost before making decisions. Its evaluation function is

f(n) = g(n) + h(n)

where

- **g(n)** = actual cost from the starting point to the current node
- **h(n)** = estimated cost from the current node to the destination

Unlike Greedy Best-First Search, A* does not simply move toward the nearest location. Instead, it evaluates both the distance already travelled and the estimated remaining distance. This balance enables the algorithm to select routes that are efficient and reliable.

In the drone delivery system, movement costs may vary because of weather conditions, strong winds, flooded roads, battery consumption, or difficult terrain. A* automatically incorporates these costs into **g(n)**.

For example, if one route is geographically shorter but passes through heavy rainfall, while another route is slightly longer but has clear weather, A* may choose the safer route because its total estimated cost is lower.

Whenever new information becomes available, such as a blocked road or severe weather warning, the algorithm recalculates the route and selects a better alternative.

Advantages

- Produces optimal paths when the heuristic is admissible.
- Consider both travel distance and travel cost.
- Adapts well to changing environments through replanning.
- Avoids unnecessary exploration.
- Provides safer navigation than Greedy Search.

Limitations

- Requires more computation.
- Consumes more memory because many nodes must be stored.
- Slightly slower than Greedy Search during decision making.

Example

A drone must choose between:

Route A

- Distance = 6 km
- Heavy rainfall
- High battery usage

Route B

- Distance = 7 km
- Clear weather
- Low battery usage

Although Route A is shorter, A* selects Route B because the overall travel cost is lower and safer.

iii. Analyze the impact of heuristic accuracy on both algorithms.

A heuristic function estimates how far a node is from the destination. The quality of this estimate significantly affects the performance of informed search algorithms.

Impact on Greedy Best-First Search

GBFS depends entirely on the heuristic.

If the heuristic accurately estimates the remaining distance, the drone quickly reaches the destination.

However, if the heuristic ignores flooded roads or weather conditions, the drone repeatedly selects poor routes, increasing delivery time and energy consumption.

Since GBFS does not consider previous travel costs, even a small heuristic error can produce inefficient paths.

Impact on A*

A* also uses the heuristic but combines it with the accumulated path cost.

Even if the heuristic contains small errors, A* often still finds a good path because **g(n)** compensates for inaccurate estimates.

When the heuristic is **admissible** (never overestimates the actual cost), A* guarantees the optimal solution.

If the heuristic overestimates the remaining distance, A* may lose optimality and choose suboptimal routes.

Comparison

Factor	Greedy Best-First Search	A* Search
Depends on heuristic	Completely	Partially
Handles inaccurate heuristic	Poorly	Better
Guarantees shortest path	No	Yes (with admissible heuristic)
Robustness	Low	High

Conclusion

The better the heuristic, the better both algorithms perform. However, A* remains far more reliable because it balances heuristic estimates with actual travel costs.

iv. Discuss how dynamic changes (blocked paths) affect search performance.

Floods, landslides, and weather changes continuously alter the search environment.

When a previously available route suddenly becomes blocked, the search algorithm must update its knowledge and compute a new path.

Effect on Greedy Best-First Search

Greedy Search immediately follows the direction that appears closest to the goal.

If that path becomes blocked, the algorithm backtracks and searches again.

This repeated backtracking increases travel time and wastes battery power.

Since it ignores previous travel cost, the drone may repeatedly choose poor alternatives.

Effect on A*

A* continuously evaluates both travelled cost and estimated remaining cost.

When an obstacle is detected, it recalculates the optimal route using updated information.

Although replanning requires extra computation, the resulting route is usually safer and more efficient.

Real-world Example

A drone begins flying toward a hospital.

After five minutes, weather updates indicate that strong winds have made one route unsafe.

Greedy Search may continue toward the dangerous area before changing direction.

A* immediately recalculates a safer path using the updated travel costs.

Overall Impact

Dynamic changes increase computational requirements for both algorithms.

However, A* handles environmental uncertainty much more effectively because it considers total travel cost rather than only estimated distance.

V. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

For emergency medicine delivery, *A Search is the most suitable algorithm**.

Although Greedy Best-First Search makes faster decisions, its inability to consider actual travel costs makes it unreliable in unpredictable environments. Emergency medical deliveries require not only speed but also safety and reliability.

A* balances actual travel cost and heuristic estimates, enabling the drone to avoid dangerous routes, conserve battery power, and reach the destination efficiently. When new obstacles appear, A* recalculates the route using updated environmental information, reducing the risk of failure.

Comparison

Criteria	Greedy Best-First Search	A* Search
Decision Speed	Very High	High
Path Optimality	Low	High
Safety	Moderate	Excellent
Handles Dynamic Changes	Poor	Very Good
Memory Usage	Low	High
Reliability	Moderate	Excellent

Final Recommendation

A* Search should be implemented in the emergency drone system because it provides the best balance between efficiency, optimality, adaptability, and safety. While it requires slightly more computation than Greedy Best-First Search, the improved route quality and ability to respond to changing conditions make it the most reliable choice for life-critical applications such as emergency medicine delivery.

SCENARIO 2: Smart Traffic Signal Optimization using AI

Problem Formulation

Traffic signal optimization is an **optimization problem** because the goal is to find the best combination of signal timings that minimizes congestion, travel time, and fuel consumption while maximizing traffic flow. Every intersection has multiple possible green-light durations, making the number of possible solutions extremely large.

Traditional search algorithms such as Breadth-First Search (BFS) and Depth-First Search (DFS) are inefficient because they attempt to explore many possible states. With hundreds of intersections, the number of combinations grows exponentially, making exhaustive search impractical.

Optimization algorithms are therefore preferred because they search for high-quality solutions without exploring every possibility.

i. Explain how Hill Climbing would work in this scenario and identify its limitations.

Hill Climbing is a local search algorithm that starts with an initial solution and continuously moves toward a neighboring solution with a better evaluation value.

In this traffic optimization problem, the initial solution may be the current traffic signal timings already in use. The algorithm evaluates traffic performance using factors such as average waiting time, queue length, fuel consumption, and vehicle throughput.

The AI slightly changes the green-light duration at one or more intersections and evaluates whether the change improves traffic flow. If the new timing reduces congestion, it becomes the new current solution. This process continues until no neighboring solution produces a better result.

Working Process

1. Start with the current signal timings.
2. Measure traffic performance.
3. Modify signal timings slightly.
4. Compare the new solution with the previous one.
5. Keep the better solution.
6. Repeat until no further improvement is found.

Advantages

- Easy to implement.
- Requires less memory.
- Produces quick improvements.
- Suitable for real-time adjustments.

Limitations

- Can stop at a local optimum instead of finding the best solution.
- Cannot easily escape flat regions (plateaus).
- Performs poorly when many traffic conditions change simultaneously.
- The final solution depends on the starting point.

Example

Suppose increasing the green signal by 5 seconds reduces waiting time at one intersection. Hill Climbing accepts this change. However, a different combination of timings across several intersections could reduce congestion even further, but the algorithm may never discover it because it only accepts immediate improvements.

ii. Discuss issues like Local Maxima, Plateaus, and Ridges with real-world interpretation.

Hill Climbing often faces three major problems that reduce its effectiveness.

1. Local Maxima

A local maximum is a solution that appears better than all nearby solutions but is not the best possible solution.

Traffic Example

A busy junction achieves good traffic flow after adjusting one signal. However, changing multiple nearby intersections together could reduce congestion much further. Since every small individual change appears worse, the algorithm stops early.

Impact

- Traffic improves only partially.
- Congestion remains during peak hours.
- Global optimization is not achieved.

2. Plateaus

A plateau is a flat region where many neighboring solutions have the same evaluation value.

Traffic Example

Changing green-light timing from 40 to 41 or 42 seconds produces almost identical waiting times.

The algorithm cannot determine which direction is better because every neighboring solution has the same performance.

Impact

- Slow progress.
- Random movement.
- Wasted computation.

3. Ridges

A ridge occurs when the best solution requires several coordinated changes simultaneously.

Traffic Example

Reducing congestion requires increasing green time at one junction while decreasing it at another. Changing only one intersection individually makes traffic worse, so Hill Climbing rejects the move.

Impact

- Unable to reach better solutions.
- Gets trapped despite the existence of superior traffic plans.

Summary Table

Problem	Meaning	Traffic Example	Effect
Local Maxima	Better than nearby solutions but not the best	Good traffic flow at one junction only	Stops early
Plateau	Equal neighboring solutions	Similar waiting times for different timings	Slow progress
Ridge	Requires multiple coordinated changes	Multiple intersections need simultaneous adjustment	Cannot reach optimal solution

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

A. Simulated Annealing

Simulated Annealing is an advanced optimization algorithm inspired by the cooling process of metals. Unlike Hill Climbing, it sometimes accepts worse solutions temporarily to escape local maxima.

Initially, the algorithm explores many different signal timing combinations. At higher temperatures, it allows occasional poor moves to avoid becoming trapped. As the temperature gradually decreases, the search becomes more focused on the best solutions.

Advantages

- Escapes local maxima.
- Handles plateaus effectively.
- Explores more of the search space.
- Produces better global solutions.

Traffic Example

Suppose increasing congestion slightly at one intersection eventually reduces congestion across the entire city. Hill Climbing rejects this temporary increase, whereas Simulated Annealing accepts it and later discovers a much better overall solution.

B. Genetic Algorithm

A Genetic Algorithm is inspired by biological evolution. Instead of improving one solution, it maintains a population of candidate traffic signal plans.

Each traffic plan is evaluated using a fitness function. The best-performing solutions are selected, combined, and slightly modified through crossover and mutation to produce new generations.

Over many generations, traffic timings improve automatically.

Advantages

- Searches many solutions simultaneously.
- Avoids local maxima.
- Handles very large search spaces.
- Produces near-optimal solutions.

Traffic Example

Suppose one traffic plan performs well during morning traffic while another works better during evening traffic. The Genetic Algorithm combines their strengths to create an improved traffic schedule.

Comparison

Feature	Hill Climbing	Simulated Annealing	Genetic Algorithm
Escapes Local Maxima	No	Yes	Yes
Explores Multiple Solutions	No	Partially	Yes
Suitable for Large Problems	Moderate	Good	Excellent
Computational Cost	Low	Medium	High
Solution Quality	Moderate	High	Very High

iv. Recommend the best approach and justify your choice based on scalability and adaptability.

Among the three algorithms, the **Genetic Algorithm (GA)** is the most suitable for optimizing traffic signals in a smart city.

Unlike Hill Climbing, which searches only one solution at a time, GA evaluates many possible traffic plans simultaneously. This enables it to handle the enormous search space created by hundreds of intersections.

GA is also highly adaptable. Traffic patterns continuously change due to accidents, weather, public events, or rush hours. By evolving solutions over successive generations, the algorithm can quickly adapt to these changes and continue producing efficient traffic signal timings.

Although Simulated Annealing improves over Hill Climbing by escaping local maxima, it still searches one solution at a time. In contrast, GA explores a diverse population of solutions, making it more effective for large-scale optimization.

Justification

- Handles large search spaces efficiently.
- Adapts to changing traffic conditions.
- Reduces waiting time and congestion.
- Minimizes fuel consumption and pollution.
- Finds high-quality near-optimal solutions.
- Suitable for real-time smart city applications.

Final Recommendation

For a city-wide intelligent traffic management system, the **Genetic Algorithm** is the best choice because it provides excellent scalability, adaptability, and optimization performance. It can efficiently manage hundreds of intersections, respond to dynamic traffic conditions, and continuously improve signal timings. While Hill Climbing is fast and simple, and Simulated Annealing improves exploration, the Genetic Algorithm offers the best balance between solution quality and real-world applicability for large smart-city traffic systems.

SCENARIO 3: Autonomous Mars Rover – Online Search Agent

i. Explain why this problem requires an Online Search Agent instead of Offline Search.

An **Online Search Agent** is more suitable because the Mars rover operates in an **unknown and dynamic environment**. Unlike offline search, where the entire environment is known before planning begins, the rover has only partial knowledge of the terrain and discovers obstacles as it moves.

The rover cannot calculate a complete path before starting because many regions of Mars have never been explored. It receives information only through its onboard sensors, cameras, and scientific instruments. Therefore, it must make decisions step by step while continuously updating its knowledge.

For example, the rover may initially choose a path toward a rock sample. While moving, it may detect a deep crater or large boulders blocking the route. Instead of following the original path, it immediately searches for a safer alternative.

An offline search algorithm would require a complete map before planning, which is impossible in this scenario. As a result, the rover would either fail to plan or generate an outdated path that ignores newly discovered obstacles.

Advantages of Online Search

- Makes decisions using real-time sensor information.
- Adapts to newly discovered obstacles.
- Suitable for unknown environments.
- Improves navigation safety.
- Allows continuous learning while exploring.

Conclusion

Since Mars is only partially explored and conditions change continuously, an **Online Search Agent** is essential for safe and intelligent navigation.

ii. Analyze the challenges of Partial Observability and Dynamic Environments.

The Mars rover operates in a **partially observable** and **dynamic** environment, making autonomous navigation a challenging AI problem.

1. Partial Observability

Partial observability means that the rover cannot see the entire environment at once. Its sensors have a limited range, and many obstacles remain hidden until the rover moves closer.

Examples

- Rocks hidden behind hills.
- Craters covered by dust.
- Slopes that cannot be detected from a distance.
- Unknown terrain beyond the camera's field of view.

Because of limited visibility, the rover must make decisions with incomplete information.

Challenges

- Increased uncertainty.
- Higher risk of choosing unsafe paths.
- Frequent replanning.
- Difficulty in predicting future conditions.

2. Dynamic Environment

Although Mars changes more slowly than Earth, the environment is still

dynamic. Examples include:

- Dust storms reducing visibility.
- Shifting sand dunes blocking routes.
- Wheel slippage on loose soil.
- Communication delays with Earth.
- Changing weather conditions affecting movement.

As the environment changes, previously safe paths may become dangerous.

Challenges

- Previously planned routes become invalid.
- Continuous path updates are required.

- Increased computational effort.
- Risk of mission failure if changes are ignored.

Overall Impact

Both partial observability and dynamic changes require the rover to continuously sense, analyze, and update its decisions. Static planning alone is insufficient for successful exploration.

iii. Describe how the rover builds and updates its internal model.

An **internal model** is the rover's stored representation of the environment. It allows the rover to remember previously visited locations and use past experiences for future decisions.

Initially, the rover has only limited information about the Martian surface. As it moves, it collects new data using cameras, LiDAR, radar, GPS (or planetary localization systems), temperature sensors, and obstacle detectors.

Each time the rover observes new terrain, it updates its internal map by recording:

- Safe paths.
- Dangerous regions.
- Rock locations.
- Craters and steep slopes.
- Sample collection points.
- Energy consumption for different terrains.

The rover also remembers unsuccessful routes to avoid repeating mistakes.

Example

Initially, the rover believes a path is safe.

After travelling, it detects loose sand that causes wheel slippage.

The rover immediately updates its internal map by marking that region as hazardous.

Future route planning automatically avoids this area.

Benefits

- Improves navigation efficiency.
- Reduces repeated exploration.
- Learns from previous experiences.

- Supports better future decisions.
- Increases mission safety.

iv. Compare Online Search with Uninformed and Informed Search

Strategies. Online Search

Online Search makes decisions while exploring the environment. It does not require complete knowledge beforehand.

Advantages

- Suitable for unknown environments.
- Learns during exploration.
- Updates plans continuously.
- Highly adaptable.

Uninformed Search

Uninformed search algorithms such as Breadth-First Search (BFS) and Depth-First Search (DFS) do not use heuristic information.

They systematically explore nodes without estimating which direction is better.

Advantages

- Simple implementation.
- Can guarantee completeness (BFS).

Disadvantages

- Explores many unnecessary states.
- High computation.
- Not practical for Mars exploration.

Informed Search

Informed search algorithms such as Greedy Best-First Search and A* use heuristic information to guide exploration.

The heuristic may estimate:

- Distance to the target.

- Energy consumption.
- Terrain difficulty.

Advantages

- Faster than uninformed search.
- Explores fewer states.
- Produces efficient routes.

Disadvantages

- Requires a good heuristic.
- Performance decreases if heuristic estimates are poor.

Comparison Table

Feature	Online Search	Uninformed Search	Informed Search
Requires Complete Map	No	Usually Yes	Usually
Yes Uses Heuristic	Optional	No	Yes
Suitable for Unknown Terrain	Excellent	Poor	Moderate
Learns During Exploration	Yes	No	Limited
Real-Time Decision Making	Excellent	Poor	Good
Computational Efficiency	High	Low	High

Analysis

For Mars exploration, Online Search is superior because it allows continuous learning and adaptation. While informed search is efficient in known environments, online search is more appropriate when the environment is unknown and constantly changing.

V. Justify the most suitable approach considering uncertainty, adaptability, and safety.

Considering the uncertainty of the Martian environment, the **Online Search Agent with a Model-Based Architecture** is the most suitable approach.

The rover cannot depend on a fixed map because it encounters new terrain throughout the mission. Instead, it must continuously gather information, update its internal model, and modify its path whenever obstacles or hazards are detected.

A model-based online agent combines real-time sensing with memory, enabling it to remember previously explored regions and avoid dangerous locations. This improves both efficiency and safety.

Compared with offline planning, the online model-based approach is more flexible and reliable because it responds immediately to environmental changes. It also minimizes unnecessary movement, conserves battery power, and increases the likelihood of successfully completing the mission.

Justification

- Handles uncertain and unknown environments.
- Learns continuously from sensor data.
- Updates routes whenever conditions change.
- Improves safety by avoiding hazards.
- Conserves energy through efficient navigation.
- Supports long-term autonomous exploration.

Final Recommendation

The **Online Search Agent with a Model-Based Agent Architecture** is the best choice for the Mars rover. It effectively handles uncertainty, partial observability, and dynamic environmental changes by continuously updating its internal model and making intelligent real-time decisions. This approach ensures safe navigation, efficient resource utilization, and successful scientific exploration, making it ideal for autonomous space missions.

SCENARIO 4: University Timetable Generation using Constraint Satisfaction Problem (CSP)

Problem Formulation as a Constraint Satisfaction Problem (CSP)

A **Constraint Satisfaction Problem (CSP)** is a problem in which a solution is obtained by assigning values to variables while satisfying all given constraints. Unlike optimization problems that search for the best solution, CSP focuses on finding a valid solution that meets every restriction.

In this scenario, the AI system must assign exam dates, time slots, and halls for every subject without violating any scheduling constraints.

The three major components of a CSP are:

- **Variables:** The items that require assignment.
- **Domains:** The possible values each variable can take.

- **Constraints:** The rules that every assignment must satisfy.

Since many courses, students, faculty members, and halls are involved, CSP is the most suitable model for solving this scheduling problem.

● Clearly define Variables, Domains, and Constraints with explanation.

1. Variables

Variables represent the decisions that the AI system must make.

In this problem, each course or subject is considered a variable because every exam requires a date, time, and examination hall.

Examples of Variables

- Mathematics Exam
- Artificial Intelligence Exam
- Data Structures Exam
- Operating Systems Exam
- Database Management Systems Exam

Each subject must receive exactly one exam schedule.

2. Domains

A domain represents the possible values that can be assigned to each variable.

For every subject, the possible values include:

- Available examination dates
- Available time slots
- Available examination halls

Example

Mathematics

Exam Possible

Dates:

- 10 June
- 11 June

- 12 June

Possible Time Slots:

- Morning Session
- Afternoon Session

Possible Halls:

- Hall A
- Hall B
- Hall C

The AI selects one valid combination from these available options

- **Constraints**

Constraints define the conditions that must always be satisfied.

Hard Constraints (Must be satisfied)

- No student should have two exams at the same time.
- Faculty should not supervise two exams simultaneously.
- Hall capacity should be sufficient for the number of students.
- Every exam must be assigned exactly one hall.
- Every exam must have one valid time slot.
- Only available faculty members should be assigned.
- Students requiring special accommodations should receive appropriate halls.

Soft Constraints (Preferred but not mandatory)

- Avoid scheduling two difficult subjects on consecutive days.
- Minimize the number of exams per student in one day.
- Schedule laboratory exams after theory exams.
- Reduce unnecessary hall changes.
- Balance faculty workload evenly

Example

Suppose:

- AI Exam → Monday Morning

- Data Science Exam → Monday Morning

If many students are enrolled in both courses, this assignment violates the "No overlapping exams" constraint.

Therefore, one exam must be moved to another time slot.

- **Explain how Backtracking Search works in this scenario.**

Backtracking Search is one of the most commonly used algorithms for solving CSPs. It assigns values to variables one at a time and checks whether the assignment satisfies all constraints.

If a constraint is violated, the algorithm goes back (backtracks) to the previous assignment and tries another possible value.

Working Process
Step 1: Select an unassigned subject.
Example:
Artificial Intelligence

Step 2: Assign a possible date, time, and hall. Example:
Monday
Mornin
g Hall A

Step 3: Check all constraints.

Questions checked include:

- Are any students writing another exam at this time?
- Is Hall A available?
- Is the faculty member available?
- Is hall capacity sufficient?

If all answers are YES, continue.

Step 4: Schedule the next subject.

Example:

Database Systems

Tuesday
Mornin
g Hall B

Again, verify all constraints.
Step 5: Continue assigning exams until all subjects are scheduled.

Step 6: If any conflict occurs, backtrack.

Example:

Operating Systems is assigned Wednesday Morning.

However, the assigned faculty is already invigilating another exam.

The algorithm removes this assignment and tries another available time slot.

Advantages

- Guarantees a valid solution if one exists.
- Simple and systematic.
- Eliminates invalid schedules early.
- Widely used for scheduling applications.

Limitations

- Slow for very large universities.
- May explore many unnecessary combinations.
- Search time increases rapidly as the number of subjects grows.

- **Discuss how Constraint Propagation (Forward Checking) improves efficiency.**

Constraint Propagation is an optimization technique that reduces the search space before conflicts occur.

One common method is **Forward Checking**.

Instead of waiting until a conflict happens, Forward Checking immediately removes impossible values from the domains of future variables.

This prevents the algorithm from exploring invalid schedules.

Example

Suppose AI Exam is scheduled on:

Monday Morning

Immediately, Forward Checking removes Monday Morning from the available options for every subject that shares students with AI.

As a result:

Before Forward Checking

Database Systems

Available Slots:

- Monday Morning
- Monday Afternoon
- Tuesday Morning

After Forward Checking

- Monday Afternoon
- Tuesday Morning

Monday Morning is removed because it would create an overlap.

Benefits

- Detects conflicts early.
- Reduces unnecessary backtracking.
- Speeds up timetable generation.
- Improves scalability.
- Produces solutions more efficiently.

Comparison

Backtracking	Forward Checking
Detects conflicts after assignment	Prevents conflicts before assignment
More backtracking	Less backtracking
Slower	Faster
Larger search space	Smaller search space

- **Analyze real-world challenges and justify the best strategy for scalability.** As universities become larger, timetable generation becomes increasingly complex. Some common real-world challenges include:

2. Large Number of Courses

Hundreds of subjects must be scheduled simultaneously.

This creates millions of possible timetable combinations.

2. Large Student Population

Students often enroll in multiple subjects.

Avoiding exam overlaps becomes increasingly difficult.

3. Faculty Availability

Faculty members may have:

- Leave
- Research commitments
- Administrative duties

The timetable must respect their availability.

4. Limited Examination Halls

Exam halls have different seating capacities.

Large classes require bigger halls.

Small classes should not waste large halls.

5. Special Accommodation Requirements

Some students require:

- Wheelchair-accessible rooms
- Extra writing time
- Separate examination halls

The timetable must accommodate these requirements.

6. Last-Minute Changes

Unexpected situations may occur:

- Faculty illness
- Hall maintenance
- Power failure
- Public holidays
- Emergency situations

The AI system must update the timetable without rebuilding everything from scratch.

Best Strategy for Scalability

For a large university, the most effective solution is to combine:

- **Backtracking Search**
- **Constraint Propagation (Forward Checking)**
- **Heuristics**, such as:
    - **Minimum Remaining Values (MRV):** Schedule the subject with the fewest available options first.
    - **Degree Heuristic:** Prioritize subjects that conflict with many others.
    - **Least Constraining Value (LCV):** Choose the assignment that leaves the maximum flexibility for remaining subjects.

This hybrid approach significantly reduces computation time and improves scheduling efficiency.

Final Recommendation

The **Constraint Satisfaction Problem (CSP)** framework combined with **Backtracking Search**, **Forward Checking**, and heuristic techniques (MRV, Degree Heuristic, and LCV) is the most suitable solution for automatic university timetable generation. This approach efficiently handles complex scheduling constraints, minimizes conflicts, and scales well as the number of courses and students increases. It provides valid, practical, and optimized timetables while remaining adaptable to real-world changes, making it ideal for modern university examination scheduling systems.

SCENARIO 5: Strategic Game AI using Minimax and Alpha-Beta Pruning

Scenario

A gaming company is developing an AI opponent for a strategy board game similar to Chess. The AI must compete against skilled human players by making intelligent decisions. Every move made by the AI influences the opponent's possible responses, creating a large game tree. Since the game has numerous possible moves, exploring every possibility is computationally expensive. Therefore, the AI must select the best move efficiently while minimizing computation time.

Problem Formulation

This problem is an **Adversarial Search Problem**, where two players compete against each other with opposite objectives.

- **Player 1 (MAX):** The AI agent tries to maximize its chances of winning.
- **Player 2 (MIN):** The human opponent tries to minimize the AI's advantage.

The AI must predict the opponent's responses before making a move. It evaluates future game states using search algorithms and chooses the move that leads to the highest possible utility while assuming the opponent also plays optimally.

● Explain how the Minimax algorithm enables the AI to make optimal decisions.

The **Minimax Algorithm** is a decision-making algorithm used in two-player, zero-sum games. It assumes that both players play optimally.

The AI explores the game tree by considering all possible future moves. At each level, the players alternate between maximizing and minimizing the score.

- **MAX Node:** Represents the AI, which selects the move with the highest utility.
- **MIN Node:** Represents the opponent, who selects the move that gives the AI the lowest utility.

The algorithm continues until it reaches terminal states (win, lose, draw) or a predefined search depth. Each terminal state is assigned a utility value, and these values are propagated upward to determine the best move.

Working Process

Step 1: Generate all possible legal moves.

Step 2: Simulate the opponent's possible responses.

Step 3: Continue expanding the game tree until a terminal state or search depth is reached.

Step 4: Evaluate each terminal state using a utility function.

Step 5: Propagate scores upward.

- MAX selects the highest score.
- MIN selects the lowest score.

Step 6: The AI chooses the move with the highest final value.

Example

Suppose the AI has three possible moves.

Move A → Utility = **+6**

Move B → Utility = **+3**

Move C → Utility = **–2**

Since the AI is the maximizing player, it selects **Move A**, which provides the highest utility.

Advantages

- Produces optimal decisions when both players play perfectly.
- Considers future consequences before making a move.
- Suitable for deterministic board games.
- Ensures rational decision-making.

Limitations

- Very slow for large game trees.
- Requires significant memory.
- Computational cost increases exponentially with search depth.

- **Discuss the computational limitations of Minimax in large game trees.**

Although Minimax guarantees optimal decisions, it becomes computationally expensive as the game complexity increases.

vi. Exponential Growth

If each game state has **b** possible moves and the search depth is **d**, the time complexity becomes:

$O(b^d)$

Even a small increase in depth greatly increases the number of nodes to evaluate.

vii. High Memory Usage

The algorithm stores many game states while exploring future possibilities.

Large games like Chess require enormous memory because millions of board positions must be evaluated.

viii. Slow Decision Making

Real-time games require quick responses.

Searching every possible move causes delays, making gameplay less interactive.

ix. Repeated Evaluation

Many branches ultimately produce the same decision, but Minimax still evaluates each one.

This results in unnecessary computation.

Real-World Example

In Chess:

- Average branching factor ≈ **35**
- Search depth = **6**

Total nodes explored:

$35^6 \approx$ **1.8 billion positions**

Evaluating such a large number of states is impractical for real-time gameplay.

Overall Impact

Without optimization, Minimax becomes inefficient for complex games because of high computation time and memory consumption.

- **Explain how Alpha-Beta Pruning improves efficiency without affecting optimality.**

Alpha-Beta Pruning is an optimization technique for the Minimax algorithm. It eliminates branches that cannot influence the final decision, thereby reducing the number of nodes evaluated.

The algorithm uses two values:

- **Alpha (α):** The best score that the MAX player can guarantee.
- **Beta (β):** The best score that the MIN player can guarantee.

Whenever $\alpha \geq \beta$, the remaining branches are pruned because they cannot improve the final outcome.

Working Process

Step 1: Begin Minimax search.

Step 2: Update Alpha at MAX nodes.

Step 3: Update Beta at MIN nodes.

Step 4: If Alpha becomes greater than or equal to Beta, stop exploring that branch.

Step 5: Continue until the best move is found.

Example

Suppose the AI already finds a move with utility **+8**.

While evaluating another branch, the opponent can force the score to **+3**.

Since **+3** is worse than **+8**, the AI ignores the remaining nodes in that branch.

Thus, unnecessary computation is avoided.

Advantages

- Evaluates far fewer nodes. Produces the same optimal decision as Minimax.

- Reduces computation time significantly.
- Enables deeper game-tree exploration.
- Improves real-time performance.

Comparison

Feature	**Minimax**	**Alpha-Beta Pruning**
Nodes Evaluated	All nodes	Only necessary nodes
Time Complexity	$O(b^d)$	Best case: $O(b^{d/2})$
Memory Usage	High	Lower
Optimal Solution	Yes	Yes
Speed	Slower	
Faster		

- **Recommend the best approach for real-time strategy games and justify your answer.**

For modern strategy games, **Minimax combined with Alpha-Beta Pruning** is the most suitable approach.

While Minimax provides intelligent and optimal decision-making, Alpha-Beta Pruning significantly improves efficiency by removing unnecessary branches. This allows the AI to search deeper within the same amount of time, resulting in stronger gameplay.

To further improve performance in commercial games, developers often combine these algorithms with:

- **Heuristic Evaluation Functions** to estimate board strength.
- **Iterative Deepening** to return the best move within a time limit.
- **Move Ordering** to maximize pruning efficiency.
- **Transposition Tables** to avoid evaluating repeated positions.

These enhancements enable the AI to make high-quality decisions even in complex game environments.

Justification

- Supports real-time gameplay.
- Handles large game trees efficiently.
- Widely used in Chess, Checkers, Othello, and other strategy games.

Final Recommendation

The best solution for developing an intelligent game-playing AI is **Minimax enhanced with Alpha-Beta Pruning**. Minimax ensures that the AI chooses the optimal move by considering the opponent's best possible responses, while Alpha-Beta Pruning improves efficiency by eliminating unnecessary branches without affecting the final decision. When combined with heuristic evaluation functions and other optimization techniques, this approach provides fast, accurate, and scalable decision-making, making it ideal for modern real-time strategy games.

SCENARIO 5: Strategic Game AI using Minimax and Alpha-Beta Pruning

Scenario

A gaming company is developing an AI opponent for a strategy board game similar to Chess. The AI must compete against skilled human players by making intelligent decisions. Every move made by the AI influences the opponent's possible responses, creating a large game tree. Since the game has numerous possible moves, exploring every possibility is computationally expensive. Therefore, the AI must select the best move efficiently while minimizing computation time.

Problem Formulation

This problem is an **Adversarial Search Problem**, where two players compete against each other with opposite objectives.

- **Player 1 (MAX):** The AI agent tries to maximize its chances of winning.
- **Player 2 (MIN):** The human opponent tries to minimize the AI's advantage.

The AI must predict the opponent's responses before making a move. It evaluates future game states using search algorithms and chooses the move that leads to the highest possible utility while assuming the opponent also plays optimally.

1.Explain how the Minimax algorithm enables the AI to make optimal decisions.

The **Minimax Algorithm** is a decision-making algorithm used in two-player, zero-sum games. It assumes that both players play optimally.

The AI explores the game tree by considering all possible future moves. At each level, the

players alternate between maximizing and minimizing the score.

- **MAX Node:** Represents the AI, which selects the move with the highest utility.
- **MIN Node:** Represents the opponent, who selects the move that gives the AI the lowest utility.

The algorithm continues until it reaches terminal states (win, lose, draw) or a predefined search depth. Each terminal state is assigned a utility value, and these values are propagated upward to determine the best move.

Working Process

Step 1: Generate all possible legal moves.

Step 2: Simulate the opponent's possible responses.

Step 3: Continue expanding the game tree until a terminal state or search depth is reached.

Step 4: Evaluate each terminal state using a utility function.

Step 5: Propagate scores upward.

- MAX selects the highest score.
- MIN selects the lowest score.

Step 6: The AI chooses the move with the highest final value.

Example

Suppose the AI has three possible moves.

Move A → Utility = **+6**

Move B → Utility = **+3**

Move C → Utility = **–2**

Since the AI is the maximizing player, it selects **Move A**, which provides the highest utility.

Advantages

- Produces optimal decisions when both players play perfectly.
- Considers future consequences before making a move.
- Suitable for deterministic board games.
- Ensures rational decision-making.

- **Discuss the computational limitations of Minimax in large game trees.**

Although Minimax guarantees optimal decisions, it becomes computationally expensive as the game complexity increases.

x. Exponential Growth

If each game state has **b** possible moves and the search depth is **d**, the time complexity becomes:

$O(b^d)$

Even a small increase in depth greatly increases the number of nodes to evaluate.

xi. High Memory Usage

The algorithm stores many game states while exploring future possibilities.

Large games like Chess require enormous memory because millions of board positions must be evaluated.

xii. Slow Decision Making

Real-time games require quick responses.

Searching every possible move causes delays, making gameplay less interactive.

xiii. Repeated Evaluation

Many branches ultimately produce the same decision, but Minimax still evaluates each one.

This results in unnecessary computation.

Real-World Example

In Chess:

- Average branching factor ≈ **35**
- Search depth = **6**

Total nodes explored:

$35^6 \approx$ **1.8 billion positions**

Evaluating such a large number of states is impractical for real-time gameplay.

Overall Impact

Without optimization, Minimax becomes inefficient for complex games because of high computation time and memory consumption.

- **Explain how Alpha-Beta Pruning improves efficiency without affecting optimality.**

Alpha-Beta Pruning is an optimization technique for the Minimax algorithm. It eliminates branches that cannot influence the final decision, thereby reducing the number of nodes evaluated.

The algorithm uses two values:

- **Alpha (α):** The best score that the MAX player can guarantee.
- **Beta (β):** The best score that the MIN player can guarantee.

Whenever $\alpha \geq \beta$, the remaining branches are pruned because they cannot improve the final outcome.

Working Process

Step 1: Begin Minimax search.

Step 2: Update Alpha at MAX nodes.

Step 3: Update Beta at MIN nodes.

Step 4: If Alpha becomes greater than or equal to Beta, stop exploring that branch.

Step 5: Continue until the best move is found.

Example

Suppose the AI already finds a move with utility **+8**.

While evaluating another branch, the opponent can force the score to **+3**.

Since **+3** is worse than **+8**, the AI ignores the remaining nodes in that branch.

Thus, unnecessary computation is avoided.

Advantages

- Evaluates far fewer nodes. Produces the same optimal decision as Minimax.

- Reduces computation time significantly.
- Enables deeper game-tree exploration.
- Improves real-time performance.

Comparison

Feature	**Minimax**	**Alpha-Beta Pruning**
Nodes Evaluated	All nodes	Only necessary nodes
Time Complexity	$O(b^d)$	Best case: $O(b^{d/2})$
Memory Usage	High	Lower
Optimal Solution	Yes	Yes
Speed	Slower	
Faster		

- **Recommend the best approach for real-time strategy games and justify your answer.**

For modern strategy games, **Minimax combined with Alpha-Beta Pruning** is the most suitable approach.

While Minimax provides intelligent and optimal decision-making, Alpha-Beta Pruning significantly improves efficiency by removing unnecessary branches. This allows the AI to search deeper within the same amount of time, resulting in stronger gameplay.

To further improve performance in commercial games, developers often combine these algorithms with:

- **Heuristic Evaluation Functions** to estimate board strength.
- **Iterative Deepening** to return the best move within a time limit.
- **Move Ordering** to maximize pruning efficiency.
- **Transposition Tables** to avoid evaluating repeated positions.

These enhancements enable the AI to make high-quality decisions even in complex game environments.

The algorithm continues until it reaches terminal states (win, lose, draw) or a predefined search depth. Each terminal state is assigned a utility value, and these values are propagated upward to determine the best move.

Working Process

Step 1: Generate all possible legal moves.

Step 2: Simulate the opponent's possible responses.

Step 3: Continue expanding the game tree until a terminal state or search depth is reached.

Step 4: Evaluate each terminal state using a utility function.

Step 5: Propagate scores upward.

- MAX selects the highest score.
- MIN selects the lowest score.

Step 6: The AI chooses the move with the highest final value.

Example

Suppose the AI has three possible moves.

Move A → Utility = **+6**

Move B → Utility = **+3**

Move C → Utility = **–2**

Since the AI is the maximizing player, it selects **Move A**, which provides the highest utility.

Advantages

- Produces optimal decisions when both players play perfectly.
- Considers future consequences before making a move.
- Suitable for deterministic board games.
- Ensures rational decision-making.

Limitations

- Very slow for large game trees.
- Requires significant memory.
- Computational cost increases exponentially with search depth.

- **Discuss the computational limitations of Minimax in large game trees.**

Although Minimax guarantees optimal decisions, it becomes computationally expensive as the game complexity increases.

xiv. Exponential Growth

If each game state has **b** possible moves and the search depth is **d**, the time complexity becomes:

$O(b^d)$

Even a small increase in depth greatly increases the number of nodes to evaluate.

xv. High Memory Usage

The algorithm stores many game states while exploring future possibilities.

Large games like Chess require enormous memory because millions of board positions must be evaluated.

xvi. Slow Decision Making

Real-time games require quick responses.

Searching every possible move causes delays, making gameplay less interactive.

xvii. Repeated Evaluation

Many branches ultimately produce the same decision, but Minimax still evaluates each one.

This results in unnecessary computation.

Real-World Example

In Chess:

- Average branching factor ≈ **35**
- Search depth = **6**

Total nodes explored:

$35^6 \approx$ **1.8 billion positions**

Evaluating such a large number of states is impractical for real-time gameplay.

Overall Impact

Without optimization, Minimax becomes inefficient for complex games because of high computation time and memory consumption.

- **Explain how Alpha-Beta Pruning improves efficiency without affecting optimality.**

Alpha-Beta Pruning is an optimization technique for the Minimax algorithm. It eliminates branches that cannot influence the final decision, thereby reducing the number of nodes evaluated.

The algorithm uses two values:

- **Alpha (α):** The best score that the MAX player can guarantee.
- **Beta (β):** The best score that the MIN player can guarantee.

Whenever **α ≥ β**, the remaining branches are pruned because they cannot improve the final outcome.

Working Process

Step 1: Begin Minimax search.

Step 2: Update Alpha at MAX nodes.

Step 3: Update Beta at MIN nodes.

Step 4: If Alpha becomes greater than or equal to Beta, stop exploring that branch.

Step 5: Continue until the best move is found.

Example

Suppose the AI already finds a move with utility **+8**.

While evaluating another branch, the opponent can force the score to **+3**.

Since **+3** is worse than **+8**, the AI ignores the remaining nodes in that branch.

Thus, unnecessary computation is avoided.

Advantages

- Evaluates far fewer nodes. Produces the same optimal decision as Minimax.

- Reduces computation time significantly.
- Enables deeper game-tree exploration.
- Improves real-time performance.

Comparison

Feature	**Minimax**	**Alpha-Beta Pruning**
Nodes Evaluated	All nodes	Only necessary nodes
Time Complexity	$O(b^d)$	Best case: $O(b^{d/2})$
Memory Usage	High	Lower
Optimal Solution	Yes	Yes
Speed	Slower	
Faster		

- **Recommend the best approach for real-time strategy games and justify your answer.**

For modern strategy games, **Minimax combined with Alpha-Beta Pruning** is the most suitable approach.

While Minimax provides intelligent and optimal decision-making, Alpha-Beta Pruning significantly improves efficiency by removing unnecessary branches. This allows the AI to search deeper within the same amount of time, resulting in stronger gameplay.

To further improve performance in commercial games, developers often combine these algorithms with:

- **Heuristic Evaluation Functions** to estimate board strength.
- **Iterative Deepening** to return the best move within a time limit.
- **Move Ordering** to maximize pruning efficiency.
- **Transposition Tables** to avoid evaluating repeated positions.

These enhancements enable the AI to make high-quality decisions even in complex game environments.

Justification

- Provides optimal decisions.
- Reduces computation time
- Supports real-time gameplay.
- Handles large game trees efficiently.

Final Recommendation

The best solution for developing an intelligent game-playing AI is **Minimax enhanced with Alpha-Beta Pruning**. Minimax ensures that the AI chooses the optimal move by considering the opponent's best possible responses, while Alpha-Beta Pruning improves efficiency by eliminating unnecessary branches without affecting the final decision. When combined with heuristic evaluation functions and other optimization techniques, this approach provides fast, accurate, and scalable decision-making, making it ideal for modern real-time strategy games